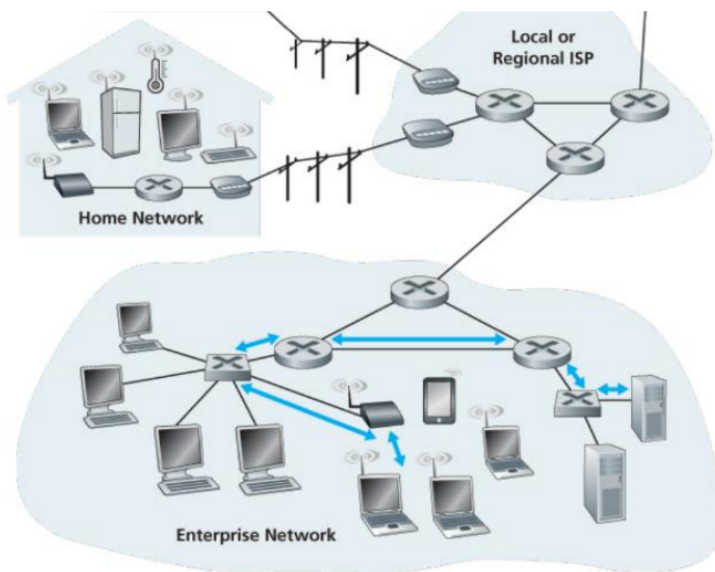


lecture 14 (Link Layer)

- Link layer:

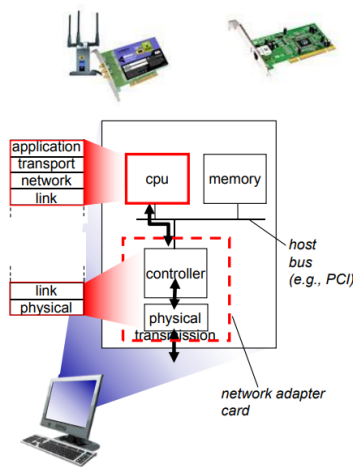
- 在网络中，凡是能发送或转发数据的设备（例如电脑、服务器、路由器）都称为“节点”**nodes**
- 链路 **links** 是节点之间的物理或无线通信通道，负责把数据从一个节点传到其“物理相邻”的另一个节点（只跨一步）。
 - 有线链路（例如光纤、双绞线）
 - 无线链路（例如 Wi-Fi、蜂窝）
- 在链路层，网络层的 **datagram**（例如 IP 包）被放入链路层帧，帧 **frame** 在物理介质上传输并包含链路层头/尾，用于邻节点之间的传输控制和差错检测。
- 链路层职责：
 - 负责把 datagram 从一个节点传到物理上相邻 **physically adjacent** 的下一个节点（即在一个链路上传输报文）
 - 链路层只保证“相邻两点”之间的传输，不负责跨多个跳的端到端传输（这是网络层/传输层的职责）。
- 不同场景下的链路类型：家庭网络（有线/无线混合）、企业网络（内部交换/无线接入）、以及连接到本地或区域 ISP 的上行链路。



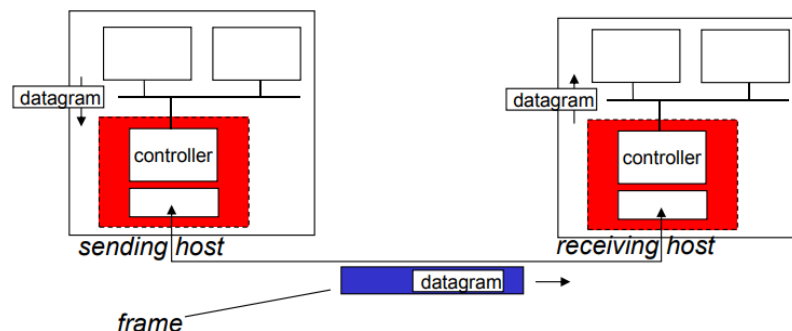
- 同一报文从源到目的地经过的每一段链路可能使用不同的物理媒介和链路协议（例如：家庭内用以太网或 Wi-Fi，中间骨干用光纤）。
- 箭头和连线表示“相邻节点之间”的链路，每条链路上运行相应的链路层协议并传送帧。
- 有的链路是共享介质（如无线局域网），有的链路是点对点（如两台路由器之间的光纤）。
 - 共享介质要考虑多站点竞争访问（需要信道访问控制），点对点则通常不需要复杂的介质访问机制。

- 链路层：context
 - 不同链路上可以使用不同的链路协议来传送同一个 datagram
 - 例：第一段链路使用以太网（Ethernet），中间链路使用 PPP（点对点协议），最后一段链路使用 802.11（Wi-Fi）。
 - 解释：IP 报文可以在每一跳被封装进该段链路对应的帧格式在物理介质上发送；接收端去除帧头尾后把 IP 报文交给上层或转发。
 - 每种链路协议能提供的服务不同
 - 例如，有的链路层协议可能提供可靠传输（rdt, reliable data transfer），有的则不提供。
 - 解释：是否在链路层做重传、差错恢复取决于该链路的特性（例如无线误码高可能需要链路重传），以及设计上的权衡（见下）。
 - 运输类比（便于理解）
 - 比喻：一次从 SUSTech 去清华的旅程可以分成多个交通段（地铁、高铁、出租），对应网络中 datagram 经过的多个链路段。
 - 旅客 = datagram（要被运送的单位）
 - 运输段 = 通信链路（相邻两站之间的一段路）
 - 运输方式 = 链路层协议（地铁/高铁/出租对应 Ethernet/PPP/802.11）
 - 旅行代理 = 路由算法（决定经过哪些节点/线路）
 - 解释：这个类比强调每段路（链路）可能的规则和条件不同，但旅客（报文）最终要被端到端送达。
- Link layer services 链路层服务
 - 帧封装与介质访问（framing, link access）
 - 将 datagram 封装成帧，加入头部和尾部（header, trailer）
 - 解释：头部通常包含目的/源 MAC 地址、类型字段等；尾部常含帧校验序列（FCS）用于差错检测。
 - 如果是共享介质，则需要信道访问控制（channel access）机制
 - 说明：常见机制包括 CSMA/CD（以太网历史上）和 CSMA/CA（无线路由中使用的冲突避免），用于管理多个设备竞争同一传输介质时的行为。
 - 帧头中使用“MAC 地址”来标识源与目的
 - 说明：MAC 地址是链路层地址，用于邻节点之间的帧转发；它与 IP 地址不同（IP 在网络层，用于端到端寻址与路由）。
 - 补充：在同一局域网内，交换机根据 MAC 地址转发帧；在跨网时，通过 ARP 等机制把 IP 地址映射到 MAC。
 - 相邻节点之间的可靠传输（reliable delivery between adjacent nodes）
 - 链路层可选择实现可靠传输（例如链路层 ACK、重传）以处理该链路上的差错。
 - 在低比特误码率的链路上（例如光纤、某些双绞线）通常不必使用链路层可靠机制

- 解释：这些物理介质很可靠，额外的链路层重传带来的开销不划算，可将可靠性留给更高层（如传输层 TCP）。
- 无线链路误码率高，常需在链路层处理更多错误（例如局部重传）
 - 说明：无线易受干扰、衰落、遮挡影响，链路层局部修复可以显著减少端到端重传延迟与开销。
- 问：为什么既要有链路层的可靠性，又要有端到端的可靠性？
 - 答：
 - 链路层可靠性能在误码率高的短链路上进行局部修复，减少端到端重传次数与时延（节省带宽和提高效率）。
 - 端到端（传输层）可靠性提供端对端语义（确保应用层交付顺序、无重复、最终到达），是跨多跳的全局保证。
 - 两者结合是为了在不同尺度（局部 vs 端到端）上权衡性能与复杂性：链路层处理物理介质产生的短时错误，传输层负责整体语义和端到端错误/丢包恢复。
 - 进一步说明：在一些系统中会避免链路层重复功能以简化（例如在非常可靠的光纤链路上不做链路重传），而在无线或不可靠链路上则启用链路重传以改善性能。
- 流量控制（flow control）
 - 控制相邻发送节点与接收节点之间的数据发送速率，以防接收方来不及处理而发生溢出。
 - 常见机制有停止等待（stop-and-wait）、滑动窗口（sliding window）等；
 - 在链路层可实现对链路上帧的速率控制，配合传输层做端到端控制。
- 差错检测（error detection）
 - 物理信号衰减或噪声会引入比特错误。
 - 接收方通过帧内的差错检测字段（如 CRC、校验和、奇偶校验）判断数据是否损坏。
 - 若检测到错误，接收方可以通知发送方重传（或直接丢弃该帧并不通知，取决于协议设计）。
- 差错纠正（error correction）
 - 接收端识别并在不进行重传的情况下纠正某些比特错误（例如使用前向纠错 FEC 技术）。
 - 纠错能减少重传次数与延迟，但会增加冗余位（开销）和复杂度。
- 半双工与全双工（half-duplex and full-duplex）
 - 半双工：链路两端节点都可以发送或接收，但不能同时发送（例如传统的无线媒介或老式串行链路）；
 - 全双工：两端可以同时双向传输（例如交换机端口上的以太网全双工）。
- 链路层通常在主机与路由器的网络接口处实现。



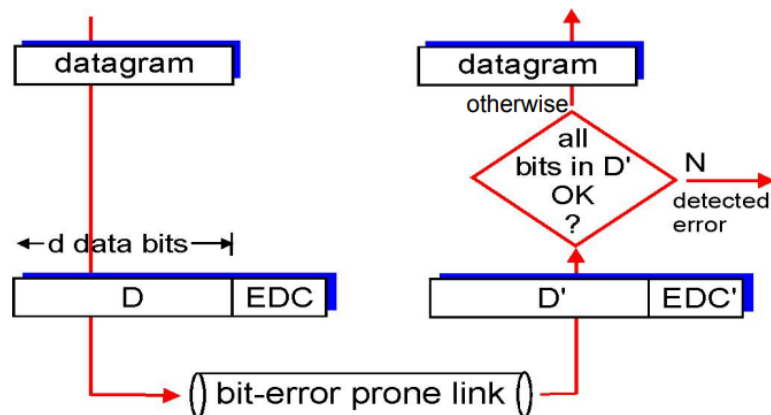
- 链路层通常在“适配器”（又称网卡，Network Interface Card，NIC）或在芯片上实现
 - 例如以太网卡、802.11 无线网卡、以太网芯片组
 - 说明：这些硬件实现了链路层和物理层的关键功能（帧封装/解析、比特定时、发送/接收电路等）。



- 发送端（sending side）
 - 将上层的 datagram 封装进帧（encapsulate），并添加差错检查位、可靠传输（rdt）相关信息、流量控制字段等。
 - 说明：封装步骤包括添加链路层头（如源/目的 MAC 地址、类型字段）和尾（如 FCS/CRC），并在必要时启动重传或序号机制。
- 接收端（receiving side）
 - 检查接收到的帧是否有错误、执行可靠传输和流量控制相关操作等；提取出 datagram 并把它交给接收端的上层（网络层）。
 - 说明：接收端可根据差错检测结果决定是否确认/重传或丢弃帧；通过驱动将有效的网络层报文传递到操作系统网络栈。
- Error detection — 差错检测
 - EDC = Error Detection and Correction bits 差错检测与纠错位
 - D = 受差错检测保护的数据（可能包含头字段）
 - 差错检测并非 100% 万无一失
 - 协议可能会漏检少数错误，但概率通常很低。
 - 增大 EDC 字段（更多冗余位）会提高检测与纠错能力，但会增加开销

(带宽/帧长度)。

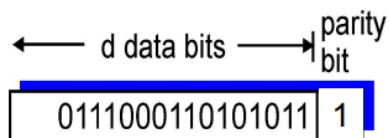
- 检测流程



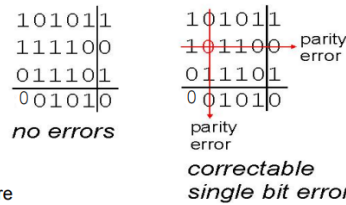
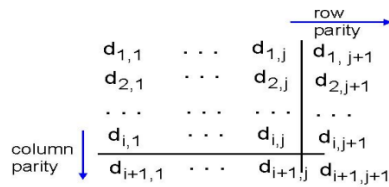
- 发送：数据 D + 生成的 EDC → 通过噪声/误码链路 → 接收端得到 D' 和 EDC' → 根据 EDC' 检查 D' 是否无错 (“all bits in D' OK?”)
 - 若检测到错误 (N)，则报告或丢弃（并视协议请求重传）；若无误则上交上层。
- 常见差错检测方法

- 奇偶校验 (Parity checks)

- 单位奇偶 (single bit parity)



- 用途：检测单比特错误（能发现奇数个位翻转）
 - 常见方案：偶校验 (even parity)，奇校验 (odd parity)
 - 说明：发送端在数据后加一个奇偶位，使得整个比特串的 1 的个数为偶数（或奇数）；接收端检查该位是否满足约定的奇偶性。
 - 只能检测奇数个翻转，偶数个翻转会被漏检（例如两个位同时翻转）。
- 二维（行列）奇偶 (two-dimensional bit parity)



more
vertical

- 功能：通过对数据按行、按列分别加奇偶位，能够检测并纠正单比特错误
- 原理：数据组织成矩阵，行尾加行奇偶位，列末加列奇偶位。若某一行与某一列的奇偶不匹配，则交点就是发生错误的单比特，可以反转纠正。
- 说明：能纠正单比特，但对多比特错误的检测/纠正能力有限（例如两个不同位置的错误可能被误判或漏检）。

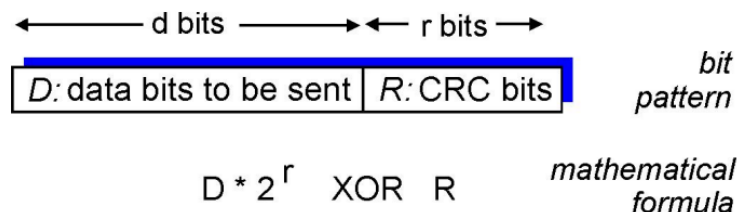
• 校验和方法（Check-summing methods）

- 把若干数据块求和并取反（或模运算），用于传输层（transport layer）UDP、TCP（虽然 TCP/UDP 的校验和强度不如 CRC）。
- 发送方（sender）：
 - 把段（segment）内容视为一串 16 位整数序列（每 16 位为一个字）。
 - 对这些 16 位字做加法（使用一位反码求和，即 one's-complement sum），得到校验和字段值。
 - “溢出回卷”（end-around carry），即超过 16 位的进位要加回低位。
 - 发送方把计算出的校验和值写入 UDP/TCP 报文头的校验和字段。
- 接收方（receiver）：
 - 对收到的段重新计算校验和
 - 将计算值与报文头中的校验和值比较：
 - 若不相等：检测到错误（NO - error detected）。
 - 若相等：未检测到错误（YES - no error detected），但仍有可能存在并未被校验和捕获的错误（checksum 不是完美的检测器）。

• 循环冗余检验（Cyclic-redundancy check, CRC）

- 链路层常用方法

- 把数据位 D 看成一个二进制多项式（或大整数），选择一个 $r+1$ 位的生成多项式（generator）G。
- 目标：
 - 计算 r 位的 CRC（记为 R），使得扩展后的位串 $\langle D, R \rangle$ 能被 G 整除（在模 2 意义下，即不带进位的二进制除法）。
 - 接收端用相同的 G 去除；若余数非零则检测到错误。
 - 若选择合适的生成多项式 G，可检测所有长度不超过 r 的连续（突发）错误（burst errors of length $\leq r$ ）；并能检测绝大多数更长错误。
 - 协议或标准会规定使用哪个 G（比如以太网、802.11、ATM 等都有固定的生成多项式）。
 - r 等于 G 的次数（degree），在位串表示上就是 G 的比特长度减 1。
 - 若 $G = 11$ （二进制，长度 2），则 $r = 1$ ；（这相当于奇偶校验）
 - 若 $G = 1011$ （长度 4），则 $r = 3$ 。
- CRC 在实践中广泛使用（例如以太网的 CRC-32、802.11 Wi-Fi、ATM 等）
- 实现要点（考试常考）：



- 计算方法概念：将 D 左移 r 位（相当于乘以 2^r ），用多项式除法除以 G，余数即 R；发送 $\langle D, R \rangle$ 。接收端除以 G，校验余数是否为 0。

CRC example

want: 即扩展后的比特串 $\langle D, R \rangle$ 恰好是 G 的整数倍

$$D 2^r \text{ XOR } R = nG$$

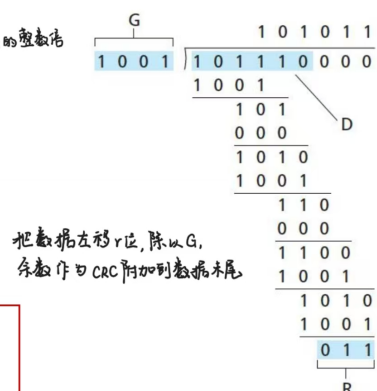
equivalently:

$$D 2^r = nG \text{ XOR } R$$

equivalently:

if we divide $D 2^r$ by G, want remainder R to satisfy:

$$R = \text{remainder} \left[\frac{D 2^r}{G} \right]$$



- 模 2 除法相当于按位异或（XOR），无需进位或借位。

- 所有 CRC 计算是在模 2 算术下进行的，计算中不带进位，减法也不带借位。
 - 加法与减法在模 2 下是相同的运算，都等价于按位异或 (XOR)。

$$\begin{array}{ll} 1011 \text{ XOR } 0101 = 1110 & 1011 - 0101 = 1110 \\ 1001 \text{ XOR } 1101 = 0100 & 1001 - 1101 = 0100 \end{array}$$

- 乘法和除法与二进制 (base-2) 算术相同，但任何需要的加或减都在“无进位/无借位”的规则下完成。
 - 在 CRC 的二进制除法里“减法”就是用 XOR 替代普通的减法，不产生进位或借位。

$$\begin{array}{r} 10001 \text{ remainder } 101 \\ 10011 \overline{)100100110} \\ \underline{10011} \\ 10110 \\ \underline{10011} \\ 101 \end{array}$$

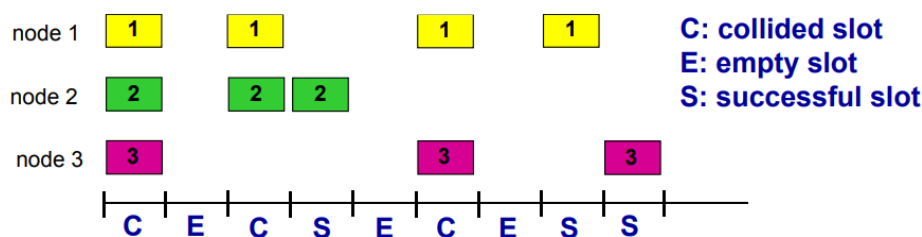
- 能检测所有单位或小于等于 r 的突发错误；对于特定生成多项式还能检测某些奇偶性/位模式导致的错误。
- 代价：需要 r 位冗余，相比简单校验开销更大，但检测能力显著提高。
- 检测 (EDC) 只能发现错误；纠错 (如 FEC、前向纠错码) 能在不重传的情况下恢复原数据，但代价是更大冗余与复杂解码。
- 将网卡插入主机的系统总线 (例如 PCI/PCIe)
 - 网卡通过主机总线与 CPU 和内存通信；有些工作由硬件 (网卡控制器) 完成，有些由驱动程序 (软件/固件) 配合完成。
- 实现方式是硬件 + 软件 + 固件的组合

• Multiple access links, protocols (多路访问链路协议)

- 定义：
 - 一个分布式算法，决定节点如何共享信道 (即谁在什么时候可以发送)
 - 用来在没有额外协调信道的情况下 (所有协调信息也需通过该共享信道发送) 实现多节点协同
 - 说明：不能依赖“带外”控制信道，协议本身必须使用该共享介质来做协调。
- 两种“链路”类型：
 - 点对点 (point-to-point)
 - 两端直接相连的链路 (例如拨号时的 PPP、两台交换机或主机之间的点对点光纤/串行链路)。
 - 广播 (共享线缆或共享介质, broadcast / shared medium)

- 多台节点共享同一物理媒介，任何节点发出的信号在媒体上对所有节点可见（例如老式共享以太网、802.11 无线 LAN、卫星链路）
- 区别点对点与广播链路的主要影响在于“介质访问控制（MAC）”的必要性：共享介质必须有 MAC 协议来协调谁什么时候发送。
- 主要情形：
 - 单一共享广播信道（single shared broadcast channel）
 - 两个或更多节点同时传输时会产生干扰（interference）
 - 若某节点在同一时间收到两个或多个信号，就称发生碰撞（collision）
 - 碰撞的出现影响吞吐量、延迟与公平性；不同 MAC 协议在这些指标上有不同权衡。
 - 协调信息（例如 RTS/CTS、ACK）也占用信道，设计时需考虑控制开销。
- 理想的多路访问协议：
 - 给定：广播信道，传输速率 R bps（比特/秒）
 - 理想协议希望满足的性质（Desired properties）：
 - 当只有一个节点想发送时，该节点能以速率 R 发送（无浪费）
 - 当有 M 个节点同时想发送时，每个节点平均能得到 R/M 的速率（公平分配）
 - 完全去中心化：无需特殊节点负责协调，不需要时钟/时隙同步
- MAC 协议分类：
 - 信道划分（channel partitioning） 信道划分 = 先划分后使用（无冲突，需要同步/分配） → TDMA/FDMA/CDMA
 - 随机接入（random access） 随机接入 = 直接发送，碰撞后重传（简单但冲突） → ALOHA/CSMA
 - 不事先划分信道，允许节点随机发送，发生冲突时再“恢复”或重传
 - 典型协议：ALOHA、Slotted ALOHA、CSMA、CSMA/CD（以太网）、CSMA/CA（Wi-Fi）
 - 优点：当节点有包要发送时可以立即以信道速率 R 发送（无需事先协调）
 - 随机接入：如果发送失败（发生碰撞），等待一段随机时间后重试
 - 两个或多个节点同时传输会发生“碰撞”（collision）
 - 随机接入 MAC 协议需定义：
 - 如何检测碰撞
 - 例如：以太网能检测到碰撞（CSMA/CD）；在无线中通常靠 ACK 超时或收不到 ACK 来判断碰撞。
 - 如何从碰撞中恢复，例如通过延迟重传或指数退避
 - Slotted ALOHA（时隙 ALOHA）
 - 假设：
 - 所有帧长度相同
 - 时间被划分成等长时隙（一个时隙等于发送一帧的时间）

- 节点仅在时隙开始处发包
- 节点是同步的
- 若某时隙有 ≥ 2 个节点同时发送，所有节点都能检测到碰撞
- 操作：
 - 节点一旦产生新帧，就在下一个时隙开始时发送
 - 若该时隙没有碰撞（no collision）→ 该节点可在下一时隙继续发下一帧（连续发）
 - 若发生碰撞（collision）→ 节点在之后的每个时隙以概率 p 重试，直到成功
 - 说明： p 通常由协议或退避算法设置，目的是降低再次碰撞概率。
- 图中展示多节点在不同时隙发送、碰撞与成功的样例序列。

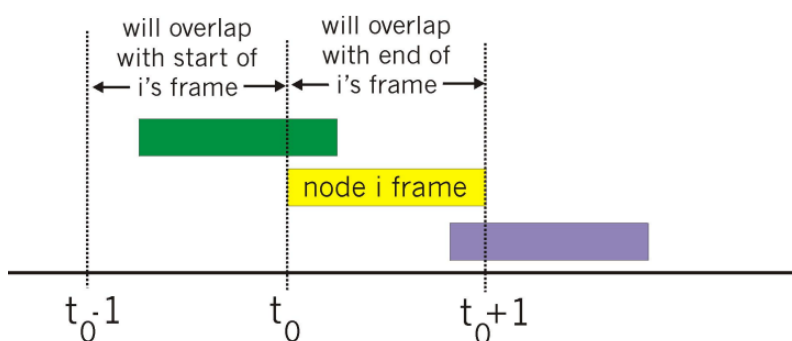


- C = 碰撞的时隙（collided slot）
- E = 空闲时隙（empty slot）
- S = 成功发送的时隙（successful slot）
- 优点：
 - 单一活跃节点可以在连续的时隙里以信道满速发送数据
 - 高度去中心化：只需保证时隙同步，没有中心协调器。
- 缺点：
 - 碰撞会导致浪费时隙
 - 空闲时隙带来资源浪费
 - 需要时钟同步，这是实现难点（尤其在分布式系统里）。
- 效率
 - 定义：长期成功时隙的比率（在很多节点且每个节点有许多帧要发的饱和条件下）。
 - 假设模型与概率：
 - 假设有 N 个节点，每个节点在每个时隙独立以概率 p 发送（或重发）。
 - 某给定节点在某时隙成功的概率 = $p * (1 - p)^{N-1}$ （它发送且其余 $N-1$ 个节点都不发送）。
 - 任意节点在某时隙成功的概率（即该时隙为成功时隙） = $N * p * (1 - p)^{N-1}$ 。
 - 要求最大效率：

- 对 p 求最大化 $N p (1-p)^{N-1}$, 得到最优 $p^* \approx 1/N$ (直觉: 每时隙期望发送节点数为 1)。
- 把 $p = 1/N$ 代入, 令 $N \rightarrow \infty$ (大节点数极限), $(1 - 1/N)^{N-1} \rightarrow 1/e$ 。
- 因此最大长期效率 = $1/e \approx 0.37$ (约 37%)。
- 结论框内强调: 信道最多只有约 37% 的时间用于有效传输, 其余时间是碰撞或空闲。
- 直观理解: 每个时隙要恰好有一个节点发送才能成功; 若过多节点导致碰撞, 过少则空闲, 最优在“平均一台发送”。

• Pure (unslotted) ALOHA (纯 ALOHA, 无时隙)

- 特点与操作:
 - 无时隙版本更简单, 不需要同步
 - 当帧到达时立即发送, 不等待时隙边界
- 碰撞概率变大:
 - 若某帧在时间 t_0 开始发送, 则它会与在区间 $[t_0 - 1, t_0 + 1]$ 内开始发送的任何帧发生碰撞。



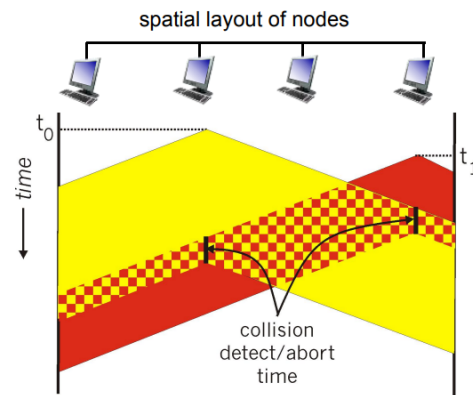
- 解释: 只要另一帧在它开始前 T 时间内或在它开始后 T 时间内开始, 都将与其重叠。换言之, 冲突窗口长度是两个帧时间 (因为另一帧的开始时间落在本帧时间前后各一个帧长度)。
- 效率:
 - 对任意给定节点在某一帧尝试成功的概率 = 节点发送的概率 $\times$ 其它节点在冲突窗口内都不发送的概率。

$$\begin{aligned}
 P(\text{success by given node}) &= P(\text{node transmits}) \cdot \\
 &\quad P(\text{no other node transmits in } [t_0-1, t_0]) \cdot \\
 &\quad P(\text{no other node transmits in } [t_0, t_0+1]) \\
 &= p \cdot (1-p)^{N-1} \cdot (1-p)^{N-1} \\
 &= p \cdot (1-p)^{2(N-1)} \\
 \dots \text{choosing optimum } p \text{ and then letting } n &\rightarrow \infty \\
 &= 1/(2e) = 0.18
 \end{aligned}$$

• CSMA (载波监听多路存取, carrier sense multiple access)

- CSMA 的基本规则是“先听后发”：
 - 若检测到信道空闲 (idle) , 可以发送整个帧。
 - 若检测到信道忙, 则推迟发送, 等待下一次机会。
- CSMA 碰撞
 - CSMA 能显著降低碰撞概率 (尤其在传播延迟很小的有线网络上) , 因为节点在发送前会检测到邻近正在发送的信号
 - 即使使用 CSMA, 碰撞仍然会发生。
 - 原因: 传播延迟 (propagation delay) 意味着两个节点可能在发送前都没听到对方刚开始的传输而认为信道空闲, 从而同时发起传输。
 - 一旦发生碰撞, 通常会浪费整个包的传输时间 —— 因为接收方无法恢复 (除非采用更复杂方法)
 - 距离与传播延迟在决定碰撞概率中起关键作用:
 - 节点之间的物理距离越大, 传播延迟越大, 发生该类不检测到的并发发送事件的可能性也越大
- CSMA/CD (碰撞检测, collision detection)
 - CSMA/CD = CSMA + CD (即在监听后发送, 同时在发送过程中继续检测是否发生碰撞)
 - 一旦检测到碰撞, 立即停止发送:
 - 碰撞会在很短时间内被检测到, 发送方可以快速中止其发送, 从而减少信道的无效占用
 - 适用场景:
 - 在有线局域网 (如早期以太网) 中很容易实现碰撞检测: 发送端能同时监测线上收到的电平或信号并将其与自己发送的信号比较, 从而发现冲突。
 - 在无线 LAN 中实现碰撞检测非常困难: 本地发送信号的强度通常会压倒接收信号, 节点在发时难以可靠侦听到其他节点的信号 (因此无线多用 CSMA/CA (避碰) 而不是 CD) 。
 -

CSMA/CD (collision detection)



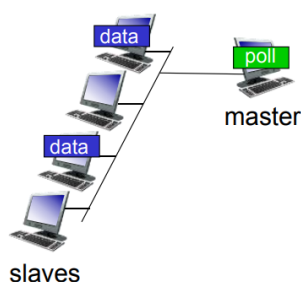
- 两个节点几乎同时开始发送，由于信号传播需要时间，两个信号会在介质上相遇并发生冲突。发送方能在一定时间内检测到冲突并中止发送。
- “碰撞检测/中止时间”就是发送方在开始发送后仍需继续监听介质的一段时间，以便在传播延迟范围内检测到来自其它节点的干扰并及时停止发送。这个时间窗口由网络的最大传播延迟决定（网络越大或传播速度越慢，窗口越长）。
- 碰撞一旦发生，能尽快检测并中止则能减少浪费的带宽（CSMA/CD 的优势之一）。
- Ethernet CSMA/CD 算法：
 - 网络适配器（NIC）从网络层接收 datagram，并将其封装成链路层帧。
 - 如果 NIC 检测到信道空闲，开始发送帧；若检测到信道忙，等待到信道空闲后再发送。
 - 如果 NIC 在发送整个帧期间未检测到其它发送（无碰撞），则发送成功完成。
 - 如果 NIC 在发送过程中检测到其它传输（碰撞），立即中止发送并发送一个“干扰/混乱（jam）”信号通知其它节点。
 - 中止后，NIC 进入二进制（指数）退避（binary exponential backoff）：
 - 第 m 次碰撞后，NIC 随机选取 $K \in \{0, 1, 2, \dots, 2^m - 1\}$ （一般对 m 有上限，例如以太网把 m 截断到 10），
 - NIC 等待 $K \times 512$ 比特时（bit times）后再次尝试（返回步骤 2），随着碰撞次数增加，最大等待窗口指数增长，从而降低再次碰撞概率。
- 轮流/占用式（“taking turns”） **轮流 = 按顺序发送或凭令牌（避免冲突，但需控制/令牌）**
→ token/polling
 - 节点轮流发送；如果某节点有更多数据可以占用更长时间
 - 典型实现：轮询（polling）、令牌环（token ring）、令牌传递（token passing）
 - 优点：能在保证公平性的同时避免冲突
 - 缺点：可能需要控制消息或令牌维护机制，延迟有时较高

- 与前两者的比较：

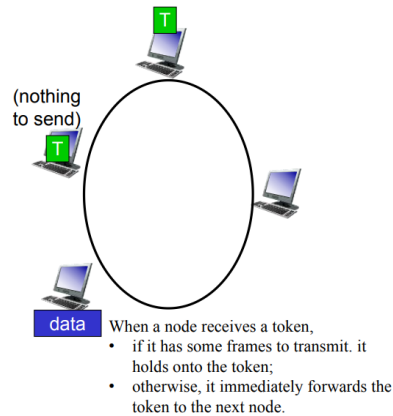
- 信道划分类 (channel partitioning) :
 - 在高负载时能高效且公平地共享信道 (每个节点有固定或分配到的资源)
 - 在低负载时效率差 (资源预留导致延迟或浪费, 例如每个节点保留 $1/N$ 带宽, 即使只有一个活跃节点也被分配固定份额)
- 随机接入类 (random access) :
 - 在低负载下效率高 (单节点可独占使用信道并以满速发送)
 - 在高负载下碰撞开销大 (吞吐下降)
- “轮流/占用”类 (taking turns) 协议:
 - 尝试兼顾两者优点:
 - 当仅一节点发送时能以速率 R 发送 (低负载优)
 - 当 M 个节点发送时平均分配为 R/M (高负载公平)

- Polling 轮询：

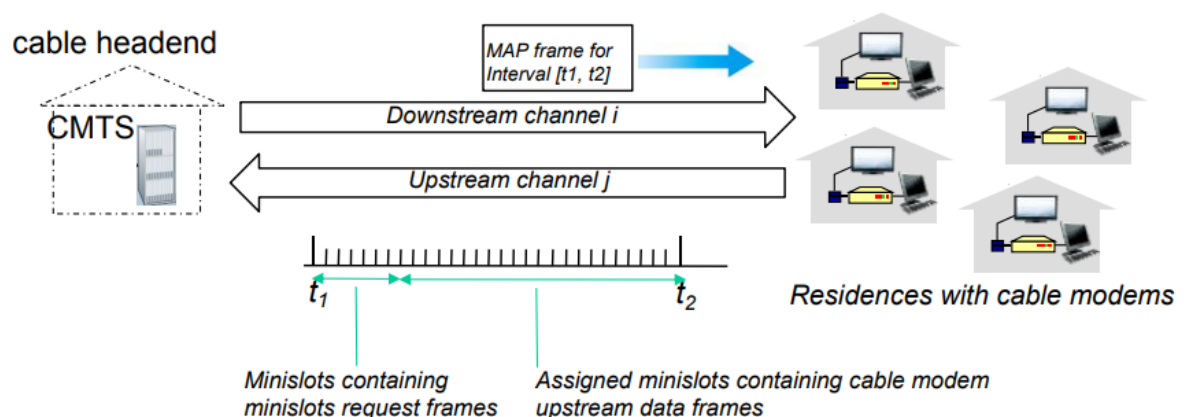
- 由一个主节点 (master) 轮流“邀请”从节点 (slave) 发送



- 被邀请的从节点可以在被允许的期间发送数据；主节点可限定每次允许的最大帧数。
 - 关注点：
 - 轮询开销 (polling overhead)：主节点需要发送轮询消息，即使某些从节点没有数据，也必须轮询到它们；控制报文消耗带宽与时间。
 - 单点故障 (single point of failure)：主节点故障会使整个轮询系统瘫痪，需考虑冗余或主备方案。
 - 优点：控制简单、无碰撞 (由主控顺序决定)，适合对延迟要求严格或需要有保证带宽的场景。
 - 缺点：对动态或大量节点不灵活，轮询延迟随节点数增长而增加；若主节点失效影响面大。
- token passing 令牌传递
 - 控制令牌按固定顺序从一个节点传到下一个节点 (顺序循环)



- 分布式 (distributed)：没有单一中心节点在发令，令牌在节点间轮转实现顺序访问。
- 关注点：
 - 令牌开销 (token overhead)：令牌本身占用少量带宽并带来延迟（尤其节点多时）。
 - 单点故障 (token 作为关键资源)：如果令牌丢失或损坏，需要有机机制重建令牌，否则环网会停滞。
- 收到令牌的节点：若有帧要发，则“持有”令牌并在允许的时间/帧数内发送；若无数据则立即把令牌转给下一个节点。
- 与轮询对比：令牌传递更分散（无中心），但仍需维护令牌管理（检测令牌丢失与重建）。
- 优点：避免碰撞、在高负载下公平、有确定性延迟
- 缺点：令牌管理与恢复复杂、令牌丢失时影响全网
- 有线接入网络 (cable access network)，用户家中布线通过同轴/光纤接入头端 (cable headend / CMTS)



- DOCSIS (Data Over Cable Service Interface Specification)：
 - 下行 (downstream) 与上行 (upstream) 使用频分 (FDM) 分离：
 - 下行为广播（无多址冲突），上行多个用户共享频带可能发生冲突。
 - 上行采用时分 (TDM) 与随机接入混合：一部分时隙由头端分配给用户（已分配的 minislot），另一部分时隙用于用户发起上行请求采用随机接入（请求里也可

能用二进制退避）。

- 头端以下行 MAP 帧通知各个调制解调器其上行时隙分配（MAP 指示哪些 minislot 可供请求或发送）。